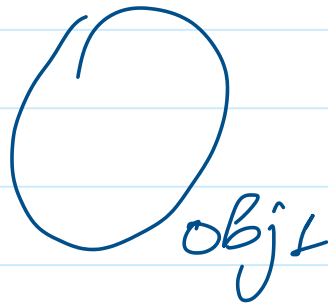


Decorator Pattern

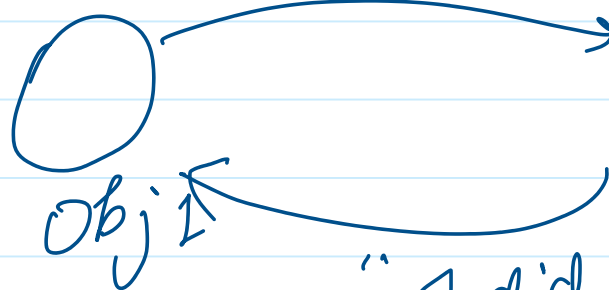
Suppose we have object



I want to give this object some extra responsibility at run time.

or
Providing additional functionality not at compile but at run time.

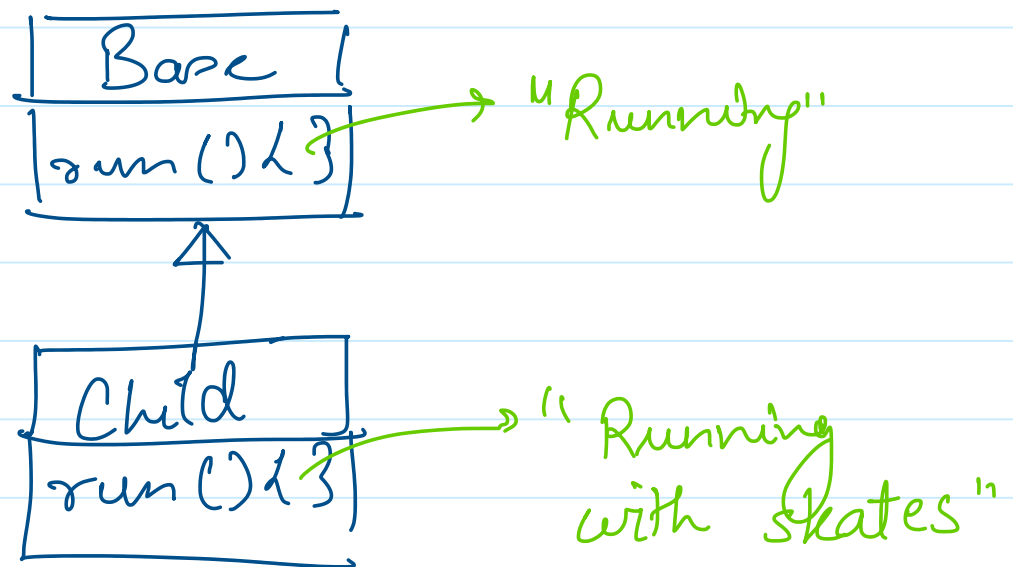
Suppose we call `doSomething()` by `obj1` and it gives us `doSomething()`



what if I can change this output dynamically

...
this output dynamically

We have Inheritance to do this.
Let's see this:-



Base *b = new
b → run()

depends on this which run() will be called.

Then why need Decorator Design Pattern?
↳ b/c Inheritance is bad as we have seen earlier.

Example :-

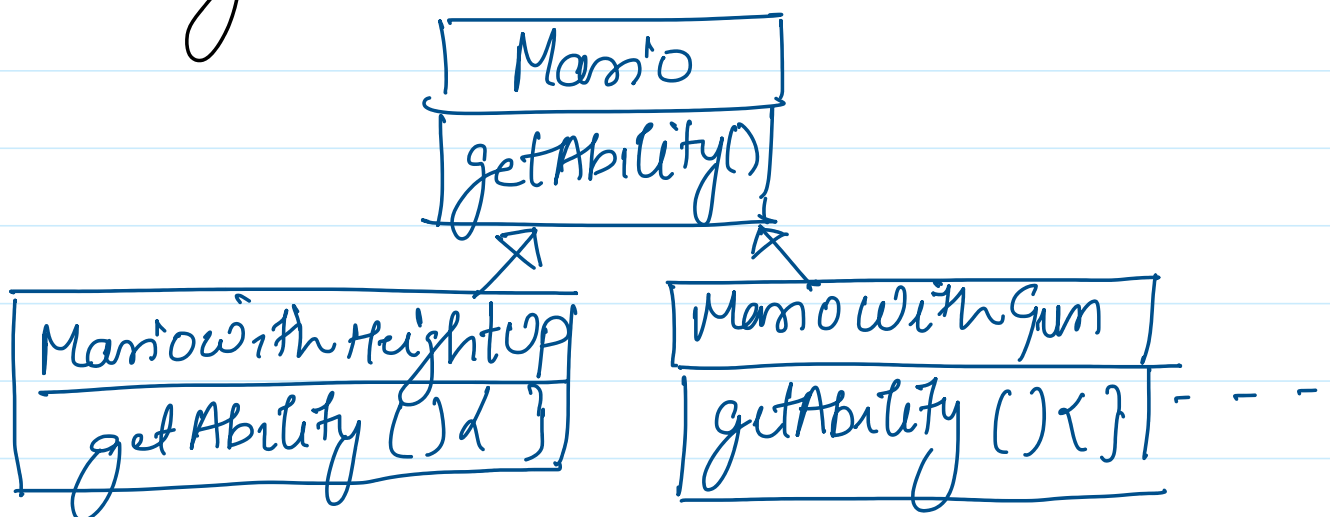
Mario Game

Mario has power ups

- ↳ Height up
- ↳ Gun shooting
- ↳ Star ability

So here we are enhancing the functionality of object by time.

Using Inheritance



Now suppose we want Mario can have Gun ability without Heightup. And again in future 'fly' ability

again in future 'fly' ability
can come so, there will more
class like :-

MarioWithFly, MarioWithFlyHeighUp,
MarioWithFlyGun - - -

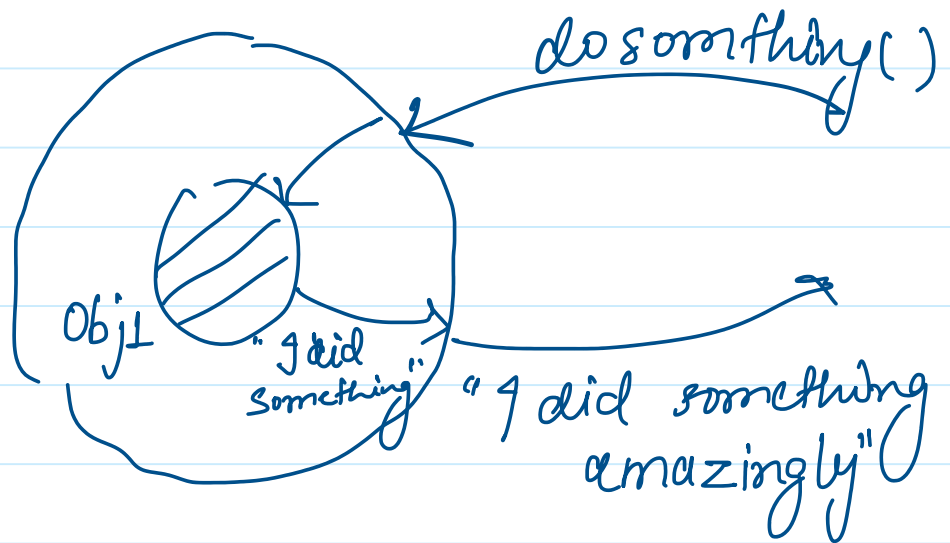
This is called Class Explosion.

↓
A new class is created after every
small change

It's difficult to manage so many
classes

So, Inheritance is BAD
Composition should be favoured above
Inheritance.

Decorator Pattern

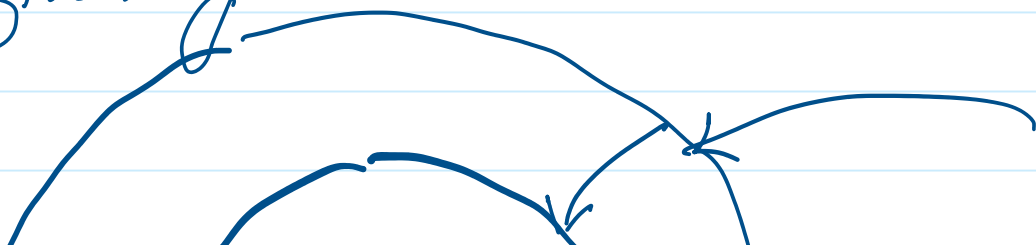


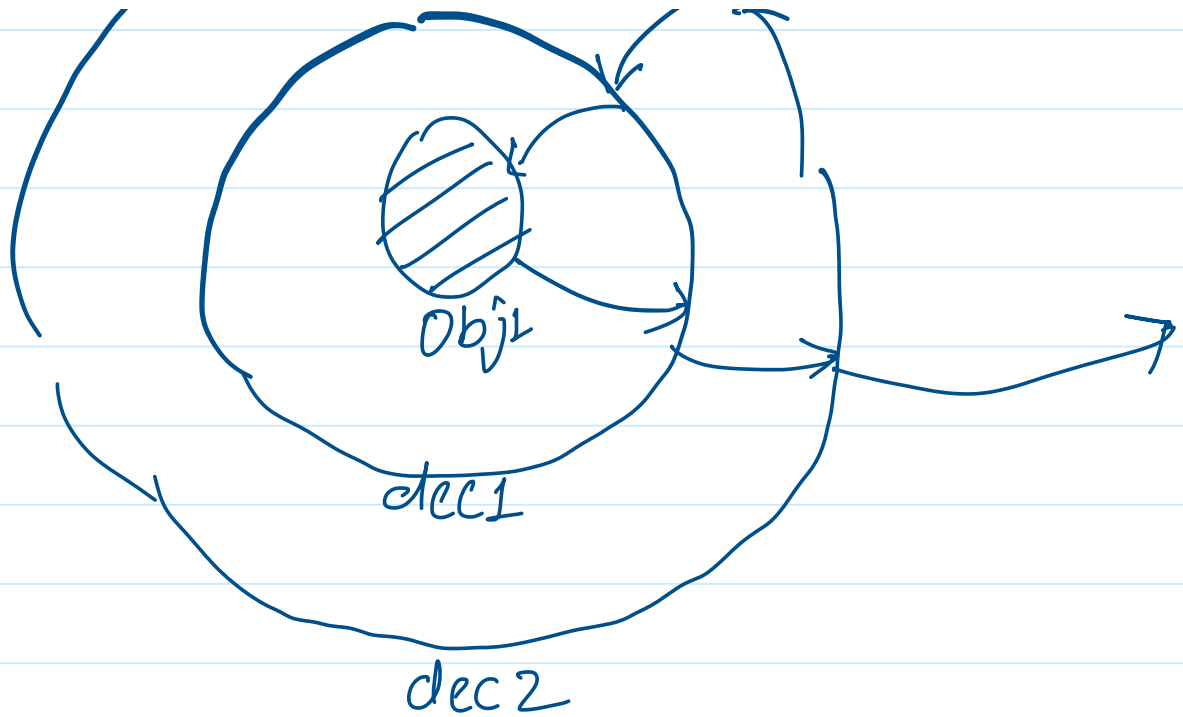
wrap `obj1` with new object `dec1`

so when you call `doSomething()`
first `dec1` call `doSomething()`
than `dec1` will call `doSomething()`
in `obj1`.

`obj1` reply to `dec1` with "I did something"
And this `dec1` can enhance this
reply and return "I did something
amazingly" to Client.

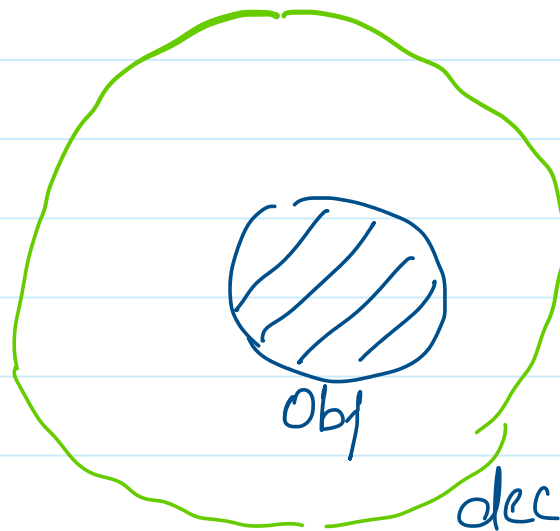
We can stack up the decorator
infinitely





So, we are enhancing their functionality without inheritance.

Now,

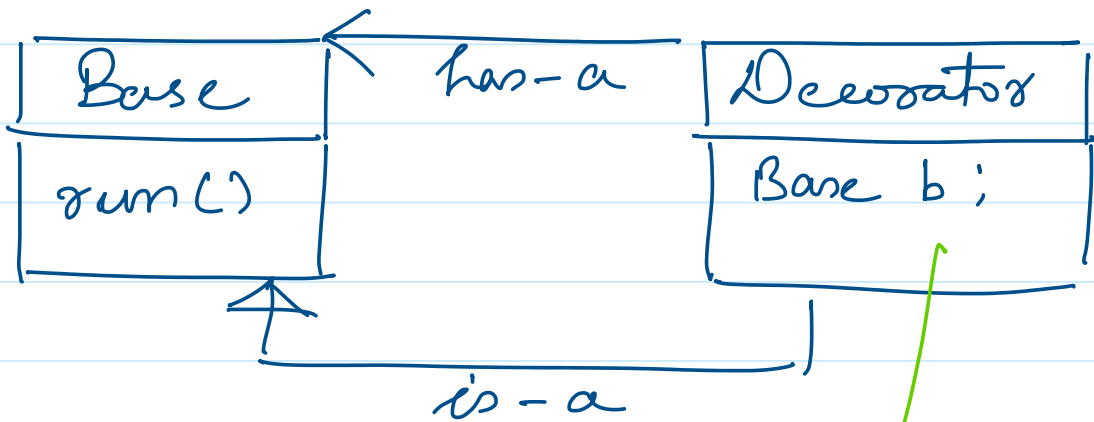


dec is also
a object

To Behave like obj, dec is definitely using Inheritance (is-a)

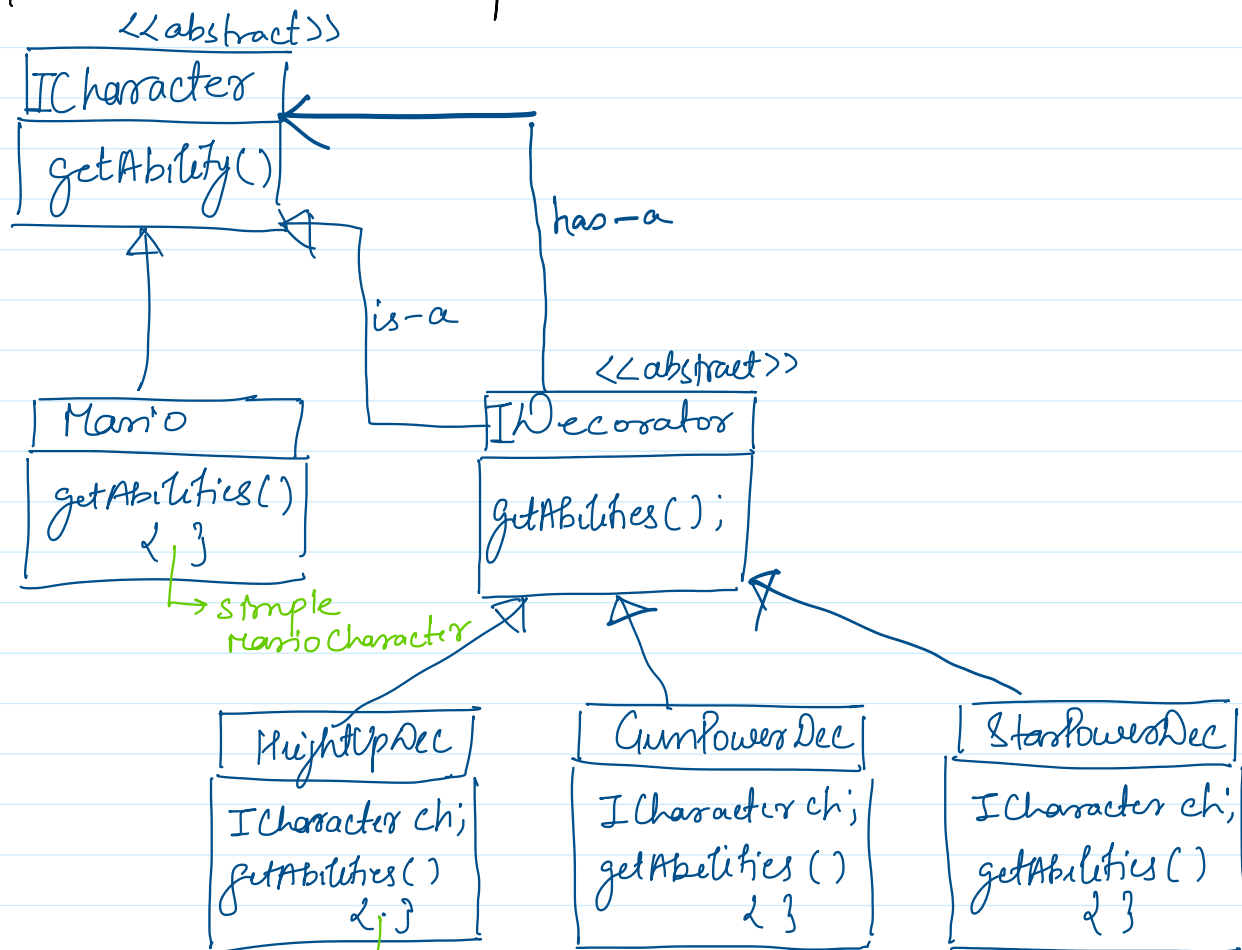
To dynamically change the behaviour decorator is using composition.

to dynamically change the behaviour
decorator is using composition
(has-a)



so it can call
Base run() and
add up its own
run()
 $b \rightarrow run() + run()$

Again go to Mario Example



Now we can add up the above 3 Classes in any order and multiple times.
No need to create new classes.

like in Inheritance

Let's create new Character:-

```
ICharacter mario = new HeightDec (new Mario ());
```

Now when we call
`mario → getAbility()`

Now when we call
mario → getAbility()

It calls the getAbility of
'HeightUp' and inside it
we can call the getAbility()
of "Mario" class.

Now we can a long chain of it as much
as we want

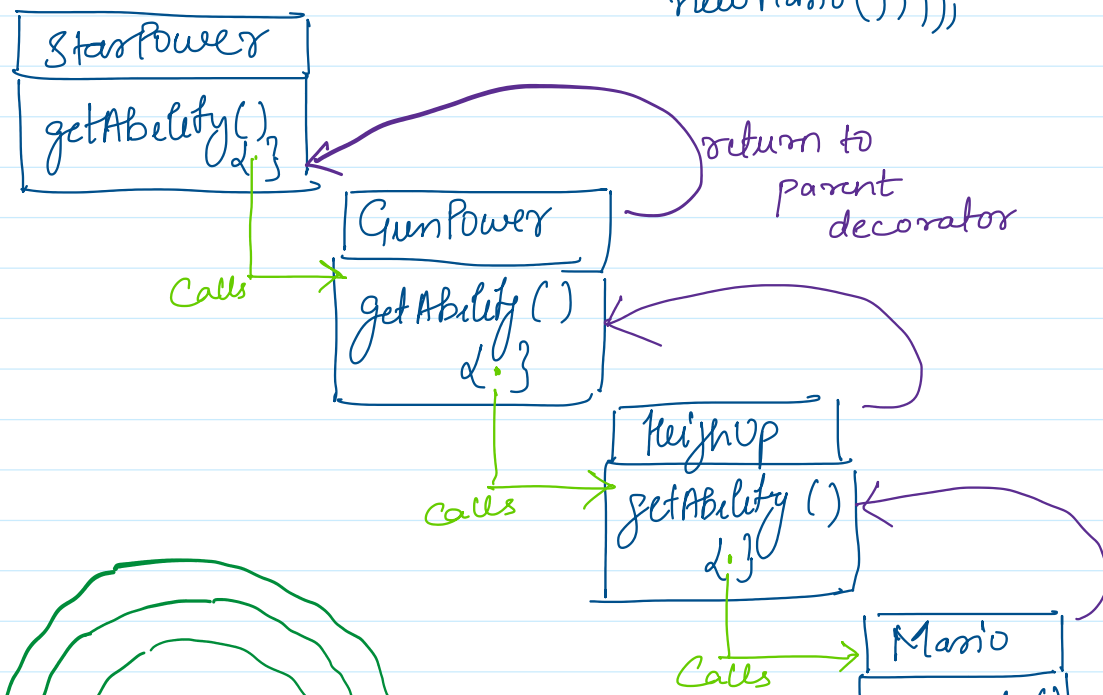
We can make as many permutation
& combination.

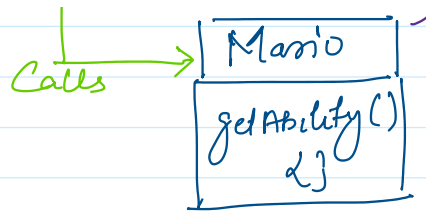
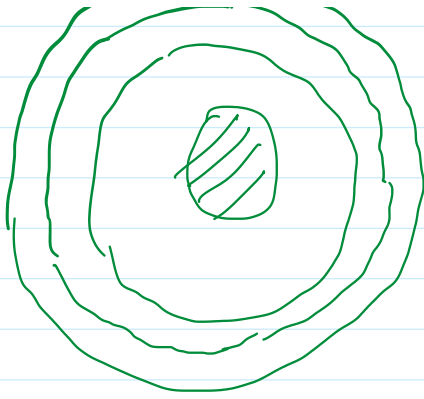
Even if 'fly' ability came we just
need to add 1 class for 'FlyDec'
No need make extra classes as we can
make it using Permutation &
combination.

So if we you thinks we are using
Recursion

Example :-

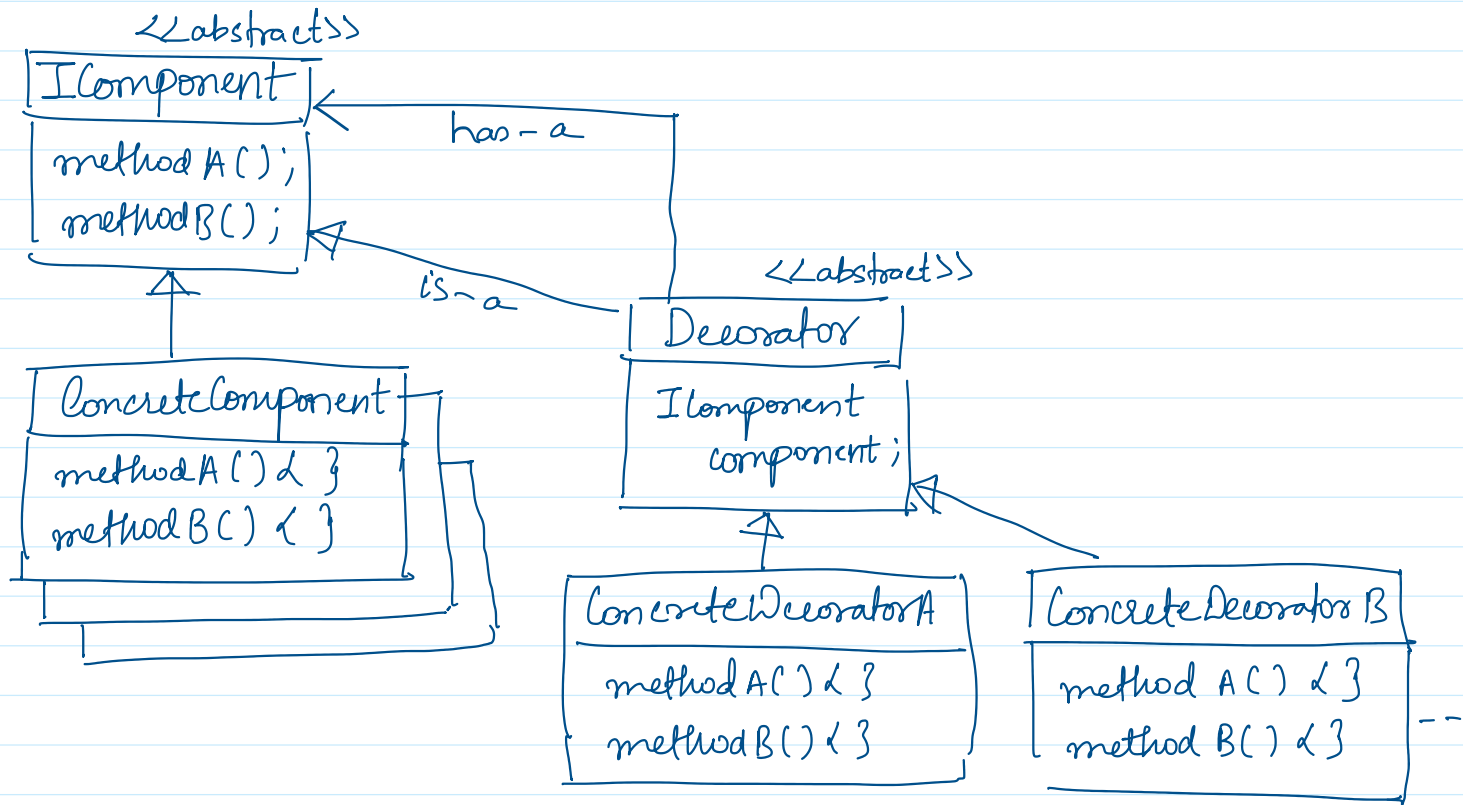
```
ICharacter mario = new StarPower(  
    new GunPower(  
        new Heightup(  
            new Mario())  
        ))  
    );
```





Here, "HeightUpDec" is also a ICharacter that's why we have a 'is-a' relationship. So, that 'GunPowerDec' can take a 'HeightUpDec' in 'ch' and we can create a chain.

Standard UML



Defination

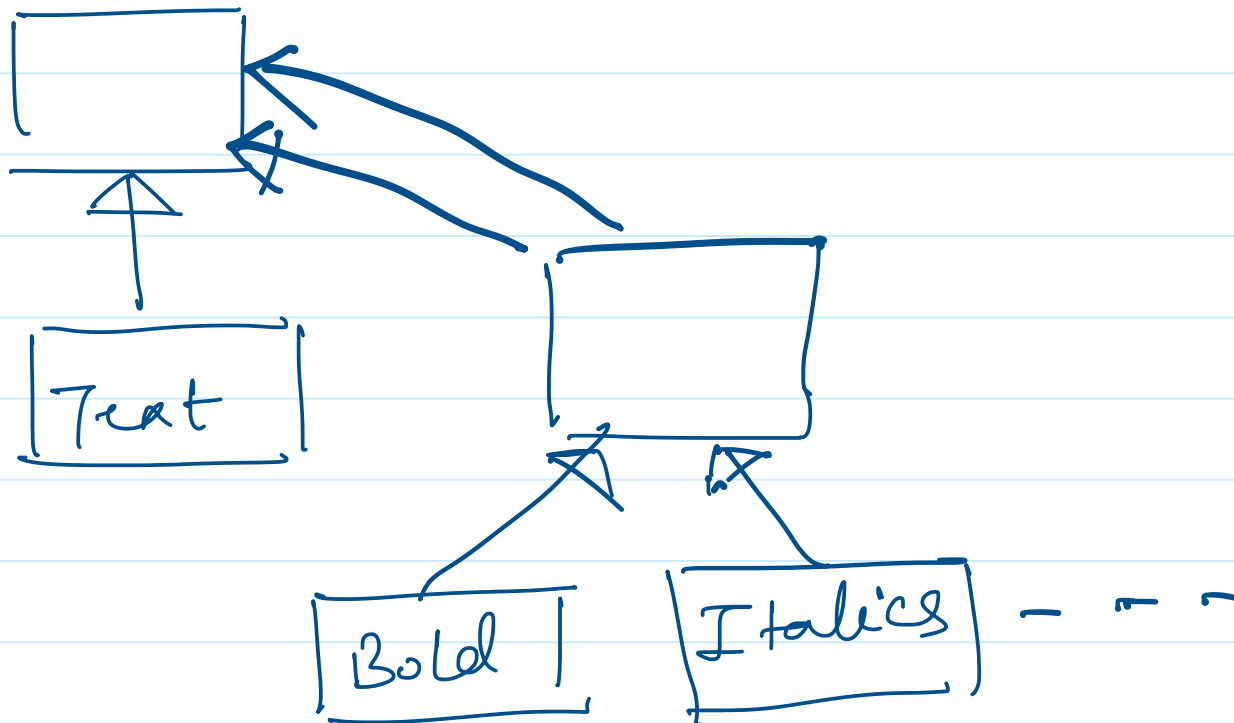
Decorator Pattern attaches additional responsibilities to an object dynamically. Decorator provides a flexible alternative to subclassing for extending functionality.

Code → see Notes. - -

Use Cases :-

1. In Docs
for and

Text we can use
Bold, **Italics**



2. form Validation

first there is a simple form and
above it we can have multiple
validation, one-above-another.